

Estructuras de Datos - Algoritmo de Dijkstra

Introduccion

El algoritmo de Dijkstra es una tecnica utilizada para resolver problemas de **caminos mas cortos** en un grafo ponderado. Es particularmente útil en redes, sistemas de transporte y otras aplicaciones donde es importante encontrar el camino mas eficiente entre nodos.

Conceptos basicos

- **Caminos mas cortos:** El objetivo es encontrar el camino con el menor costo (o peso) entre dos vertices.
- **Pesos:** Cada arista tiene un peso que puede representar distancia, tiempo u otra metrica relevante.
- **Funcionamiento general:**
 - Parte de un vertice inicial V_0 y calcula las menores distancias hacia todos los demas vertices.
 - Utiliza un conjunto de distancias D para representar los costos minimos desde V_0 hasta cada vertice.
 - Itera seleccionando el vertice con la menor distancia no evaluada.

Pasos del algoritmo

1. Inicializacion:

- Establecer D con infinito para todos los vertices excepto el inicial, que tiene distancia 0.
- Crear una cola de prioridad para gestionar los vertices según sus distancias minimas.

2. Iteracion:

- Extraer el vertice con la menor distancia desde la cola.
- Marcar el vertice como evaluado.
- Actualizar las distancias a sus vecinos si se encuentra un camino mas corto pasando por este vertice.

3. Finalizacion:

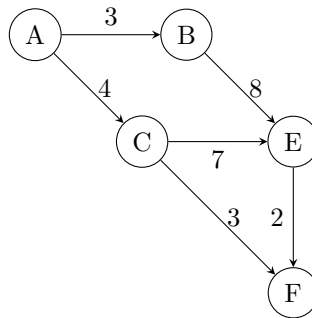
- Repetir el proceso hasta que todos los vertices hayan sido evaluados o la cola este vacia.

Ejemplo practico

Dado el siguiente grafo:

- **Vertices:** A, B, C, D, E, F .
- **Aristas:**
 - $A \rightarrow B$: peso 3.
 - $A \rightarrow C$: peso 4.
 - $B \rightarrow E$: peso 8.
 - $C \rightarrow E$: peso 7.
 - $C \rightarrow F$: peso 3.
 - $E \rightarrow F$: peso 2.

Representacion Grafica:



Al aplicar el algoritmo de Dijkstra desde el vertice A :

- **Distancias iniciales:** $D[A] = 0$, $D[B] = 3$, $D[C] = 4$, $D[D] = \infty$, $D[E] = 8$, $D[F] = \infty$.
- **Iteracion:**
 - Evaluar B : no mejora las distancias.
 - Evaluar C : actualiza $D[E] = 7$ y $D[F] = 10$.
 - Evaluar E : mejora $D[F] = 9$.

Complejidad del Algoritmo

- **Tiempo:** Con una cola de prioridad basada en un monticulo (heap), el tiempo es $O((V + E) \log V)$, donde V es el número de vertices y E es el número de aristas.
- **Espacio:** Requiere almacenamiento para V vertices y E aristas.

Codigo del Algoritmo en Java

Implementacion del Algoritmo de Dijkstra:

```
1 import java.util.*;
2
3 class Grafo {
4     private Map<String, List<Edge>> adyacencia = new HashMap<>();
5
6     // Metodo para agregar un vertice al grafo
7     public void agregarVertice(String vertice) {
8         adyacencia.putIfAbsent(vertice, new ArrayList<>());
9     }
10
11     // Metodo para agregar una arista con peso entre dos vertices
12     public void agregarArista(String origen, String destino, int peso) {
13         adyacencia.get(origen).add(new Edge(destino, peso));
14     }
15
16     // Implementacion del algoritmo de Dijkstra
17     public Map<String, Integer> dijkstra(String inicio) {
18         // Cola de prioridad para procesar vertices por distancia minima
19         PriorityQueue<Edge> cola = new PriorityQueue<>(Comparator.comparingInt(e ->
20             e.peso));
21         // Mapa para almacenar las distancias mas cortas desde el inicio
22         Map<String, Integer> distancias = new HashMap<>();
23
24         // Inicializar las distancias a infinito, excepto para el vertice inicial
25         for (String vertice : adyacencia.keySet()) {
26             distancias.put(vertice, Integer.MAX_VALUE);
27         }
28         distancias.put(inicio, 0);
29         cola.add(new Edge(inicio, 0));
30
31         // Procesar la cola de prioridad
32         while (!cola.isEmpty()) {
33             Edge actual = cola.poll(); // Extraer el vertice con la menor distancia
34             String nodo = actual.destino;
35
36             // Explorar los vecinos del nodo actual
37             for (Edge vecino : adyacencia.get(nodo)) {
38                 // Calcular la nueva distancia a traves del nodo actual
39                 int nuevaDistancia = distancias.get(nodo) + vecino.peso;
40                 if (nuevaDistancia < distancias.get(vecino.destino)) {
41                     distancias.put(vecino.destino, nuevaDistancia); // Actualizar la
42                         distancia
43                     cola.add(new Edge(vecino.destino, nuevaDistancia)); // Agregar
44                         el vecino a la cola
45                 }
46             }
47         }
48         return distancias; // Devolver las distancias mas cortas
49     }
50
51     // Clase interna para representar una arista
52     static class Edge {
53         String destino;
54         int peso;
55     }
56 }
```

```

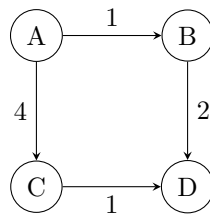
52     Edge(String destino, int peso) {
53         this.destino = destino;
54         this.peso = peso;
55     }
56 }
57
58
59 public static void main(String[] args) {
60     Grafo grafo = new Grafo();
61     // Agregar vertices al grafo
62     grafo.agregarVertice("A");
63     grafo.agregarVertice("B");
64     grafo.agregarVertice("C");
65     grafo.agregarVertice("E");
66     grafo.agregarVertice("F");
67
68     // Agregar aristas con sus respectivos pesos
69     grafo.agregarArista("A", "B", 3);
70     grafo.agregarArista("A", "C", 4);
71     grafo.agregarArista("B", "E", 8);
72     grafo.agregarArista("C", "E", 7);
73     grafo.agregarArista("C", "F", 3);
74     grafo.agregarArista("E", "F", 2);
75
76     // Ejecutar el algoritmo de Dijkstra desde el vertice "A"
77     Map<String, Integer> distancias = grafo.dijkstra("A");
78     // Imprimir las distancias mas cortas
79     System.out.println("Distancias desde A: " + distancias);
80 }
81 }

```

Ejercicio 1: Facil

Enunciado: Resuelve el camino mas corto desde el vertice *A* a todos los demas vertices en el siguiente grafo no dirigido y ponderado.

Grafo:



Resolucion:

- **Paso 1: Inicializacion.** Establecer distancias iniciales:

$$- D[A] = 0, D[B] = \infty, D[C] = \infty, D[D] = \infty.$$

- **Paso 2: Procesar vertice A.** Actualizar distancias para vecinos de A:

– $D[B] = 1$, $D[C] = 4$.

- **Paso 3: Procesar vertice B.** Actualizar distancia para D :

– $D[D] = 3$.

- **Paso 4: Procesar vertice C.** No mejora las distancias.
- **Resultado final:** $D[A] = 0$, $D[B] = 1$, $D[C] = 4$, $D[D] = 3$.

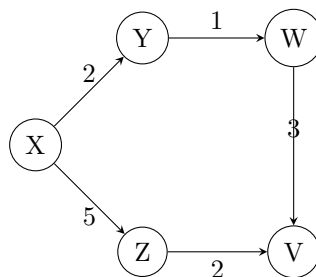
Codigo en Java:

```
1 public static void main(String[] args) {
2     Grafo grafo = new Grafo();
3
4     // Agregar vertices
5     grafo.agregarVertice("A");
6     grafo.agregarVertice("B");
7     grafo.agregarVertice("C");
8     grafo.agregarVertice("D");
9
10    // Agregar aristas
11    grafo.agregarArista("A", "B", 1);
12    grafo.agregarArista("A", "C", 4);
13    grafo.agregarArista("B", "D", 2);
14    grafo.agregarArista("C", "D", 1);
15
16    // Ejecutar Dijkstra
17    Map<String, Integer> distancias = grafo.dijkstra("A");
18    System.out.println("Distancias desde A: " + distancias);
19 }
```

Ejercicio 2: Intermedio

Enunciado: Encuentra el camino mas corto desde el vertice X en el siguiente grafo dirigido y ponderado.

Grafo:



Resolucion:

- **Paso 1: Inicializacion.** $D[X] = 0$, $D[Y] = \infty$, $D[Z] = \infty$, $D[W] = \infty$, $D[V] = \infty$.
- **Paso 2: Procesar vertice X.** $D[Y] = 2$, $D[Z] = 5$.

- Paso 3: Procesar vertice Y. $D[W] = 3$.
- Paso 4: Procesar vertice Z. $D[V] = 7$.
- Paso 5: Procesar vertice W. $D[V] = 6$.
- Resultado final: $D[X] = 0$, $D[Y] = 2$, $D[Z] = 5$, $D[W] = 3$, $D[V] = 6$.

Codigo en Java:

```

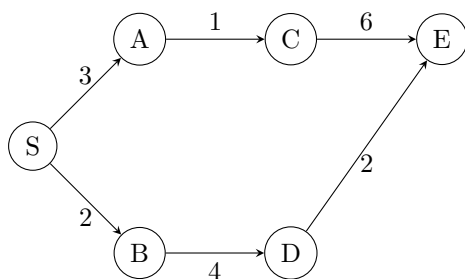
1 public static void main(String[] args) {
2     Grafo grafo = new Grafo();
3
4     // Agregar vertices
5     grafo.agregarVertice("X");
6     grafo.agregarVertice("Y");
7     grafo.agregarVertice("Z");
8     grafo.agregarVertice("W");
9     grafo.agregarVertice("V");
10
11    // Agregar aristas
12    grafo.agregarArista("X", "Y", 2);
13    grafo.agregarArista("X", "Z", 5);
14    grafo.agregarArista("Y", "W", 1);
15    grafo.agregarArista("Z", "V", 2);
16    grafo.agregarArista("W", "V", 3);
17
18    // Ejecutar Dijkstra
19    Map<String, Integer> distancias = grafo.dijkstra("X");
20    System.out.println("Distancias desde X: " + distancias);
21 }

```

Ejercicio 3: Difícil

Enunciado: Determina los caminos mas cortos desde el vertice S en el siguiente grafo con multiples conexiones y pesos variables.

Grafo:



Resolucion:

- Paso 1: Inicializacion. $D[S] = 0$, $D[A] = \infty$, $D[B] = \infty$, $D[C] = \infty$, $D[D] = \infty$, $D[E] = \infty$.

-
- Paso 2: Procesar vertice S. $D[A] = 3$, $D[B] = 2$.
 - Paso 3: Procesar vertice B. $D[D] = 6$.
 - Paso 4: Procesar vertice A. $D[C] = 4$.
 - Paso 5: Procesar vertice D. $D[E] = 8$.
 - Paso 6: Procesar vertice C. No mejora las distancias.
 - Resultado final: $D[S] = 0$, $D[A] = 3$, $D[B] = 2$, $D[C] = 4$, $D[D] = 6$, $D[E] = 8$.

Codigo en Java:

```
1 public static void main(String[] args) {
2     Grafo grafo = new Grafo();
3
4     // Agregar vertices
5     grafo.agregarVertice("S");
6     grafo.agregarVertice("A");
7     grafo.agregarVertice("B");
8     grafo.agregarVertice("C");
9     grafo.agregarVertice("D");
10    grafo.agregarVertice("E");
11
12    // Agregar aristas
13    grafo.agregarArista("S", "A", 3);
14    grafo.agregarArista("S", "B", 2);
15    grafo.agregarArista("A", "C", 1);
16    grafo.agregarArista("B", "D", 4);
17    grafo.agregarArista("C", "E", 6);
18    grafo.agregarArista("D", "E", 2);
19
20    // Ejecutar Dijkstra
21    Map<String, Integer> distancias = grafo.dijkstra("S");
22    System.out.println("Distancias desde S: " + distancias);
23 }
```